

Script 1: Inicialización de la base de datos de la Pokédex local

Este script se encarga de crear la estructura necesaria en un archivo `pokemon.db` para luego almacenar la información descargada desde la PokéAPI. Está pensado para ejecutarse **una sola vez** antes de la descarga masiva de datos.

¿Qué hace el script?

1. Se conecta al archivo `pokemon.db` (lo crea si no existe).
2. Activa la verificación de claves foráneas (`PRAGMA foreign_keys = ON`), obligatoria para que las relaciones entre tablas se respeten.
3. Crea todas las tablas necesarias:
 - `tipos`: guarda los nombres únicos de los tipos de Pokémon.
 - `pokemon`: almacena los datos básicos de cada Pokémon (id, nombre, altura, peso, experiencia base).
 - `pokemon_tipo`: tabla intermedia para la relación muchos a muchos entre Pokémon y tipos.
 - `log_nuevos`: tabla de auditoría donde un trigger registrará automáticamente cada nuevo Pokémon insertado.
4. Crea índices para acelerar las consultas más frecuentes:
 - Índice sobre `nombre` en la tabla `pokemon` (búsquedas por nombre).
 - Índice sobre `tipo_id` en `pokemon_tipo` (búsquedas de Pokémon por tipo).
5. Crea una vista llamada `vista_pokemon_tipos` que muestra cada Pokémon con sus tipos concatenados en una sola columna, lista para usar en consultas sin escribir JOINS cada vez.
6. Crea un trigger `trg_nuevo_pokemon` que inserta automáticamente un registro en `log_nuevos` cada vez que se añade un Pokémon.

Todas las sentencias usan `IF NOT EXISTS` o `DROP TRIGGER IF EXISTS` para que el script se pueda ejecutar varias veces sin provocar errores.

Código completo

A continuación se muestra el script en Python con comentarios detallados. Dentro del propio código se ejecutan las instrucciones SQL usando el módulo `sqlite3` de la biblioteca estándar.

```
import sqlite3
import os

# -----
# 1. Conexión a la base de datos (se crea si no existe)
# -----
# Si el archivo pokemon.db no existe, sqlite3 lo crea automáticamente.
# Usamos el administrador de contexto 'with' para asegurar que la conexión
# se cierre correctamente al finalizar.
with sqlite3.connect("pokemon.db") as conexion:
```

```

# -----
# 2. Activación de claves foráneas
# -----
# Sin esta línea, las restricciones FOREIGN KEY se ignoran.
# Debe ejecutarse en cada nueva conexión.
conexion.execute("PRAGMA foreign_keys = ON")

# Obtenemos un cursor para ejecutar las sentencias
cursor = conexion.cursor()

# -----
# 3. Creación de tablas
# -----
# Usamos CREATE TABLE IF NOT EXISTS para que el script sea idempotente.

# Tabla de tipos de Pokémon
cursor.execute("""
    CREATE TABLE IF NOT EXISTS tipos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT UNIQUE NOT NULL
    )
""")

# Tabla principal de Pokémon
cursor.execute("""
    CREATE TABLE IF NOT EXISTS pokemon (
        id INTEGER PRIMARY KEY,
        nombre TEXT NOT NULL,
        altura REAL,
        peso REAL,
        experiencia_base INTEGER
    )
""")

# Tabla intermedia para la relación muchos a muchos
cursor.execute("""
    CREATE TABLE IF NOT EXISTS pokemon_tipo (
        pokemon_id INTEGER NOT NULL,
        tipo_id INTEGER NOT NULL,
        PRIMARY KEY (pokemon_id, tipo_id),
        FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
        FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
    )
""")

# Tabla de auditoría para registrar cada nuevo Pokémon insertado
cursor.execute("""
    CREATE TABLE IF NOT EXISTS log_nuevos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        pokemon_id INTEGER NOT NULL,
        fecha TEXT DEFAULT (datetime('now')),
        FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE
    )
""")

```

```

    )
    """

# -----
# 4. Creación de índices
# -----
# Mejoran el rendimiento de las consultas por nombre y por tipo.

# Índice para búsquedas por nombre de Pokémon
cursor.execute("""
    CREATE INDEX IF NOT EXISTS idx_pokemon_nombre ON pokemon (nombre)
""")

# Índice para obtener todos los Pokémon de un tipo concreto
cursor.execute("""
    CREATE INDEX IF NOT EXISTS idx_pokemon_tipo_tipo ON pokemon_tipo
(tipo_id)
""")

# -----
# 5. Creación de la vista
# -----
# "vista_pokemon_tipos" devuelve cada Pokémon con sus tipos
# concatenados (separados por coma) en una sola columna.
cursor.execute("""
    CREATE VIEW IF NOT EXISTS vista_pokemon_tipos AS
    SELECT p.id, p.nombre,
           GROUP_CONCAT(t.nombre, ', ') AS tipos
    FROM pokemon p
    JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
    JOIN tipos t ON pt.tipo_id = t.id
    GROUP BY p.id
""")

# -----
# 6. Creación del trigger
# -----
# Para que el script sea reejecutable sin errores, primero
# eliminamos el trigger si ya existía y luego lo creamos.
cursor.execute("DROP TRIGGER IF EXISTS trg_nuevo_pokemon")

cursor.execute("""
    CREATE TRIGGER trg_nuevo_pokemon
    AFTER INSERT ON pokemon
    FOR EACH ROW
    BEGIN
        INSERT INTO log_nuevos (pokemon_id) VALUES (NEW.id);
    END
""")

# -----
# 7. Confirmación de los cambios
# -----

```

```
# Al tratarse de un bloque 'with', el commit se hace automáticamente
# si no hay excepciones. Pero por claridad podemos llamarlo también:
conexion.commit()

print("Base de datos 'pokemon.db' inicializada correctamente.")
```

Cómo utilizar este script

1. Guarda el código anterior en un archivo, por ejemplo `inicializar_db.py`.
2. Ejecútalo con Python 3 desde la terminal:

```
python inicializar_db.py
```

3. Se creará (o actualizará) el archivo `pokemon.db` en el mismo directorio.
4. Ya estará lista para recibir los datos de la PokéAPI con el siguiente script de descarga.

Notas importantes

- **Conexión a internet:** no se necesita, porque este script solo define estructuras.
- **Seguridad de ejecución múltiple:** todas las sentencias utilizan `IF NOT EXISTS` o un `DROP` previo, por lo que puedes ejecutarlo cuantas veces quieras sin perder datos existentes.
- **Integridad referencial:** el `PRAGMA foreign_keys = ON` asegura que las relaciones se respeten en cualquier operación futura.
- **Fecha automática en auditoría:** la columna `fecha` de `log_nuevos` se rellena con la fecha/hora actual en el momento de la inserción, gracias a la función `datetime('now')`.

Este es el primer paso hacia una Pokédex local totalmente funcional sin depender de internet.

Script 2: Descarga de TODOS los Pokémon desde la PokéAPI

Este script se conecta a la **PokéAPI pública** para obtener **todos los Pokémon disponibles**

¿Qué hace el script?

1. Establece la conexión a la base de datos local (previamente creada con el Script 1).
2. Descubre el número total de Pokémon disponibles preguntando a la API (`/pokemon?limit=1`).
3. Define la función `guardar_pokemon` que inserta tipos, Pokémon y relaciones (idéntica a la versión anterior, idempotente).
4. Itera desde el ID 1 hasta el total, solicitando cada Pokémon a `https://pokeapi.co/api/v2/pokemon/{id}`.
5. Gestiona errores de red, datos mal formados o Pokémon inexistentes (puede haber huecos, aunque la API suele devolver 404 para IDs no asignados).

6. Incluye una pausa configurable entre peticiones para no saturar el servicio público.
7. Al finalizar, confirma todos los cambios y cierra la conexión.

El script es **idempotente**: si se interrumpe, puede reanudarse sin duplicar datos, ya que utiliza **INSERT OR IGNORE**. Podría ejecutarse de nuevo para completar los Pokémon que faltaran.

Código completo

```
import sqlite3
import requests
import time

# -----
# 1. Conexión a la base de datos local
# -----
with sqlite3.connect("pokemon.db") as conexion:
    conexion.execute("PRAGMA foreign_keys = ON")
    cursor = conexion.cursor()

# -----
# 2. Función de inserción de un Pokémon
# -----
def guardar_pokemon(conexion, datos_json):
    #Almacena un Pokémon y sus tipos en la base de datos.Ignota inserciones
    #duplicadas (idempotente).
    cursor = conexion.cursor()

    poke_id = datos_json["id"]
    nombre = datos_json["name"].lower()
    altura = datos_json["height"]
    peso = datos_json["weight"]
    experiencia = datos_json.get("base_experience")

    tipos_nombres = [t["type"]["name"] for t in datos_json["types"]]

    # Insertar tipos (si no existen)
    for tipo_nombre in tipos_nombres:
        cursor.execute(
            "INSERT OR IGNORE INTO tipos (nombre) VALUES (?)",
            (tipo_nombre,)
        )

    # Insertar Pokémon
    cursor.execute(
        """INSERT OR IGNORE INTO pokemon
        (id, nombre, altura, peso, experiencia_base)
        VALUES (?, ?, ?, ?, ?)""",
        (poke_id, nombre, altura, peso, experiencia)
    )

    # Insertar relaciones en pokemon_tipo
```

```

        for tipo_nombre in tipos_nombres:
            cursor.execute("SELECT id FROM tipos WHERE nombre = ?",
(tipo_nombre,))
            fila = cursor.fetchone()
            if fila is None:
                cursor.execute("INSERT INTO tipos (nombre) VALUES (?)",
(tipo_nombre,))
                tipo_id = cursor.lastrowid
            else:
                tipo_id = fila[0]

            cursor.execute(
                "INSERT OR IGNORE INTO pokemon_tipo (pokemon_id, tipo_id)
VALUES (?, ?)",
                (poke_id, tipo_id)
            )

# -----
# 3. Obtener el número total de Pokémon disponibles
# -----
print("Consultando el número total de Pokémon en la API...")
try:
    resp = requests.get("https://pokeapi.co/api/v2/pokemon?limit=1")
    resp.raise_for_status()
    total_pokemon = resp.json()["count"]
    print(f"Se van a descargar {total_pokemon} Pokémon.")
except Exception as e:
    print(f"No se pudo obtener el total de Pokémon: {e}")
    print("Se usará un número alto por defecto (2000).")
    total_pokemon = 2000 # Valor de respaldo

# -----
# 4. Descarga y almacenamiento de cada Pokémon
# -----
pausa = 0.3 # segundos entre peticiones (ajustable)

for pokemon_id in range(1, total_pokemon + 1):
    url = f"https://pokeapi.co/api/v2/pokemon/{pokemon_id}"

    try:
        respuesta = requests.get(url, timeout=10)
        if respuesta.status_code == 404:
            # La API no tiene un Pokémon con ese ID (es normal que haya
huecos)
            print(f" [{pokemon_id}/{total_pokemon}] No existe (404). Se
omite.")
            continue
        respuesta.raise_for_status()

        datos = respuesta.json()
        guardar_pokemon(conexion, datos)
        print(f" [{pokemon_id}/{total_pokemon}] {datos['name']}
guardado.")

```

```

except requests.exceptions.RequestException as e:
    print(f" [{pokemon_id}/{total_pokemon}] Error de red: {e}")
except Exception as e:
    print(f" [{pokemon_id}/{total_pokemon}] Error inesperado: {e}")

time.sleep(pausa)

# -----
# 5. Finalización
# -----
conexion.commit()
print("¡Descarga finalizada! Todos los Pokémon disponibles han sido
guardados en 'pokemon.db'.")

```

Notas sobre el funcionamiento

- **Total dinámico:** El script consulta primero <https://pokeapi.co/api/v2/pokemon?limit=1> y extrae el campo `count` (actualmente alrededor de 1300). Esto garantiza que siempre se intentarán descargar todos los Pokémon que existen.
- **Huecos en la numeración:** No todas las IDs consecutivas tienen un Pokémon. Si la API devuelve 404, el script lo indica y continúa sin problemas.
- **Tiempo de ejecución:** Con una pausa de 0.3 segundos y más de 1000 Pokémon, el proceso puede durar varios minutos. Es normal.
- **Idempotencia:** Gracias a `INSERT OR IGNORE`, si el script se detiene (por Ctrl+C, error de red, etc.), puedes volver a ejecutarlo y retomará la descarga omitiendo los que ya existen.
- **Dependencias:** Necesitas la biblioteca `requests`. Instálala con `pip install requests` si no la tienes.
- **Base de datos previa:** Asegúrate de haber ejecutado el Script 1 (`inicializar_db.py`) antes de lanzar este, para que las tablas existan.

Posibles ajustes

- Si quieres limitar a una generación concreta, puedes cambiar `range(1, total_pokemon + 1)` por el rango deseado.
- La pausa `pausa = 0.3` se puede aumentar si encuentras errores de límite de tasa (la API pública es tolerante, pero mejor ser respetuoso).
- Para descargas futuras de mantenimiento, basta con volver a ejecutar el script: solo añadirá Pokémon nuevos (si la API ha crecido).

Con este segundo script, tu base de datos local contendrá la Pokédex completa, lista para ser consultada sin conexión.

Script 3: Consultas offline con fallback a la PokéAPI

Este script está pensado para ser el **punto de entrada interactivo** de tu Pokédex local. Una vez que la base de datos `pokemon.db` contiene los Pokémon descargados con el Script 2, puedes realizar distintas consultas **sin conexión a internet**. Sin embargo, si en algún momento la base de datos no existe, está vacía o no encuentra el Pokémon solicitado, el programa automáticamente recurre a la PokéAPI para obtener la información en tiempo real.

¿Qué hace el script?

1. Intenta conectarse al archivo `pokemon.db`. Si no existe, informa que la base de datos no está disponible y que se usará la API como respaldo.
2. Muestra un **menú interactivo** con las siguientes opciones:
 - **1. Buscar Pokémon por nombre o ID:** Muestra datos básicos y tipos. Si no está en local, pregunta a la API.
 - **2. Listar Pokémon de un tipo:** Usa la tabla local o la API pública `/type/{nombre}`.
 - **3. Top 10 Pokémon más pesados:** Solo disponible si hay base de datos local (requiere ordenar por peso).
 - **4. Altura media de todos los Pokémon:** Solo con base de datos local (cálculo agregado).
 - **5. Exportar Pokédex a CSV:** Solo con base de datos local (vuelca la vista `vista_pokemon_tipos`).
 - **6. Salir.**
3. Cada función que interactúa con la base de datos comprueba primero si la conexión es válida. Si no lo es, deriva a la API cuando es posible o muestra un mensaje de funcionalidad no disponible.
4. Utiliza consultas parametrizadas y maneja posibles errores de red.

Así, el programa resulta útil tanto si ya has poblado tu base de datos como si solo quieres hacer una consulta rápida a la API, actuando como un cliente universal de la Pokédex.

Requisitos

- Python 3.
- Bibliotecas estándar (`sqlite3`, `csv`, `os`) y la biblioteca externa `requests`.
Instálala con `pip install requests` si no la tienes.
- Para las opciones 3, 4 y 5 es imprescindible tener una base de datos local con datos.
Las opciones 1 y 2 funcionan siempre, incluso sin base de datos.

Código completo

```
import sqlite3
import requests
import os
import csv

# -----
# 1. Conexión a la base de datos local
# -----
def conectar_db():
    """
```



```

Intenta abrir la base de datos pokemon.db.
Retorna una conexión válida o None si el archivo no existe.
"""
if not os.path.exists("pokemon.db"):
    print("Aviso: no se encuentra 'pokemon.db'. Se trabajará solo con la
API.")
    return None
try:
    conexion = sqlite3.connect("pokemon.db")
    conexion.execute("PRAGMA foreign_keys = ON")
    return conexion
except Exception as e:
    print(f"Error al conectar con la base de datos: {e}")
    return None

# -----
# 2. Funciones auxiliares de consulta
# -----
def buscar_local(conexion, nombre_o_id):
    """
    Busca un Pokémon en la base de datos local por nombre o ID.
    Devuelve un diccionario con los datos o None si no se encuentra.
    """
    cursor = conexion.cursor()
    # Intentamos como ID numérico
    try:
        poke_id = int(nombre_o_id)
        cursor.execute("""
            SELECT id, nombre, altura, peso, experiencia_base
            FROM pokemon WHERE id = ?
        """, (poke_id,))
        fila = cursor.fetchone()
        if fila:
            # Obtener tipos
            cursor.execute("""
                SELECT GROUP_CONCAT(t.nombre, ', ')
                FROM pokemon_tipo pt
                JOIN tipos t ON pt.tipo_id = t.id
                WHERE pt.pokemon_id = ?
            """, (poke_id,))
            tipos = cursor.fetchone()[0] or "sin tipo"
            return {
                "id": fila[0],
                "nombre": fila[1],
                "altura": fila[2],
                "peso": fila[3],
                "experiencia_base": fila[4],
                "tipos": tipos
            }
        except ValueError:
            pass

    # Si no es ID, buscamos por nombre exacto o parcial

```

```

cursor.execute("""
    SELECT id, nombre, altura, peso, experiencia_base
    FROM pokemon
    WHERE LOWER(nombre) = LOWER(?) OR LOWER(nombre) LIKE ?
""", (nombre_o_id, f"%{nombre_o_id}%"))
results = cursor.fetchall()
if results:
    # Si hay múltiples coincidencias, mostramos la primera
    fila = results[0]
    cursor.execute("""
        SELECT GROUP_CONCAT(t.nombre, ', ')
        FROM pokemon_tipo pt
        JOIN tipos t ON pt.tipo_id = t.id
        WHERE pt.pokemon_id = ?
""", (fila[0],))
    tipos = cursor.fetchone()[0] or "sin tipo"
    return {
        "id": fila[0],
        "nombre": fila[1],
        "altura": fila[2],
        "peso": fila[3],
        "experiencia_base": fila[4],
        "tipos": tipos
    }
return None

def buscar_api(nombre_o_id):
    """
    Busca un Pokémon en la PokéAPI por nombre o ID.
    Retorna un diccionario con los datos formateados o None si no existe.
    """
    url = f"https://pokeapi.co/api/v2/pokemon/{nombre_o_id.lower()}"
    try:
        resp = requests.get(url, timeout=10)
        resp.raise_for_status()
        datos = resp.json()
        return {
            "id": datos["id"],
            "nombre": datos["name"].lower(),
            "altura": datos["height"],
            "peso": datos["weight"],
            "experiencia_base": datos.get("base_experience"),
            "tipos": ", ".join(t["type"]["name"] for t in datos["types"])
        }
    except requests.exceptions.RequestException as e:
        print(f"Error de conexión a la API: {e}")
    except Exception as e:
        print(f"Error al procesar la respuesta de la API: {e}")
    return None

# -----
# 3. Funciones del menú
# -----

```

```

def opcion_buscar(conexion):
    nombre = input("Nombre o ID del Pokémon: ").strip()
    if not nombre:
        print("Debe introducir un valor.")
        return

    pokemon = None
    if conexion:
        pokemon = buscar_local(conexion, nombre)
        if pokemon:
            print("(Obtenido de la base de datos local)")

    if not pokemon:
        print("No encontrado en local. Consultando en la API...")
        pokemon = buscar_api(nombre)
        if not pokemon:
            print("No se encontró el Pokémon ni en local ni en la API.")
            return

    print(f"\n ID:           {pokemon['id']}")
    print(f" Nombre:        {pokemon['nombre'].capitalize()}")
    print(f" Tipos:         {pokemon['tipos']}")
    print(f" Altura:        {pokemon['altura']} dm ({pokemon['altura']/10}
m)")
    print(f" Peso:          {pokemon['peso']} hg ({pokemon['peso']/10} kg)")
    if pokemon['experiencia_base'] is not None:
        print(f" Exp. base:     {pokemon['experiencia_base']}")
    else:
        print(" Exp. base:     --")

def opcion_tipo(conexion):
    tipo = input("Introduce un tipo (ej. fire, water, electric):
").strip().lower()
    if not tipo:
        return

    if conexion:
        # Consulta local con JOIN
        cursor = conexion.cursor()
        cursor.execute("""
            SELECT p.nombre
            FROM pokemon p
            JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
            JOIN tipos t ON pt.tipo_id = t.id
            WHERE LOWER(t.nombre) = ?
            ORDER BY p.nombre
            """, (tipo,))
        results = cursor.fetchall()
        if results:
            print(f"\nPokémon de tipo '{tipo}' (desde BD local):")
            for (nombre,) in results:
                print(f" - {nombre.capitalize()}")
            print(f"Total: {len(results)}")

```

```

        return
    else:
        print("Tipo no encontrado en la base de datos local.")

# Fallback a API
print("Consultando tipo en la API...")
url = f"https://pokeapi.co/api/v2/type/{tipo}"
try:
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()
    data = resp.json()
    pokemon_list = [p["pokemon"]["name"] for p in data["pokemon"]]
    if pokemon_list:
        print(f"\nPokémon de tipo '{tipo}' (desde API):")
        for nombre in pokemon_list:
            print(f"    - {nombre.capitalize()}")
        print(f"Total: {len(pokemon_list)}")
    else:
        print("No se encontraron Pokémon de ese tipo.")
except requests.exceptions.RequestException as e:
    print(f"Error al conectar con la API: {e}")
except Exception as e:
    print(f"Error al procesar la respuesta: {e}")

def opcion_top10_pesados(conexion):
    if not conexion:
        print("Opción no disponible sin base de datos local.")
        return
    cursor = conexion.cursor()
    cursor.execute("""
        SELECT nombre, peso
        FROM pokemon
        WHERE peso IS NOT NULL
        ORDER BY peso DESC
        LIMIT 10
    """)
    results = cursor.fetchall()
    print("\nTop 10 Pokémon más pesados:")
    for nombre, peso in results:
        print(f"    {nombre.capitalize():<20} {peso} hg ({peso/10} kg)")

def opcion_altura_media(conexion):
    if not conexion:
        print("Opción no disponible sin base de datos local.")
        return
    cursor = conexion.cursor()
    cursor.execute("SELECT AVG(altura) FROM pokemon WHERE altura IS NOT NULL")
    media = cursor.fetchone()[0]
    if media:
        print(f"\nAltura media de todos los Pokémon: {media:.2f} dm
        ({media/10:.2f} m)")
    else:
        print("No se pudo calcular la altura media.")

```

```

def opcion_exportar_csv(conexion):
    if not conexion:
        print("Opción no disponible sin base de datos local.")
        return
    archivo = input("Nombre del archivo CSV (p.ej. pokedex.csv): ").strip()
    if not archivo:
        archivo = "pokedex.csv"
    cursor = conexion.cursor()
    cursor.execute("SELECT * FROM vista_pokemon_tipos")
    with open(archivo, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["ID", "Nombre", "Tipos"])
        writer.writerows(cursor.fetchall())
    print(f"Pokédex exportada a '{archivo}' correctamente.")

# -----
# 4. Menú principal
# -----
def main():
    print("=== Pokédex Interactiva (Local + API) ===")
    conexion = conectar_db()

    while True:
        print("\n--- Menú ---")
        print("1. Buscar Pokémon por nombre o ID")
        print("2. Listar Pokémon de un tipo")
        print("3. Top 10 Pokémon más pesados")
        print("4. Altura media de todos los Pokémon")
        print("5. Exportar Pokédex a CSV")
        print("6. Salir")
        opcion = input("Elige una opción: ").strip()

        if opcion == "1":
            opcion_buscar(conexion)
        elif opcion == "2":
            opcion_tipo(conexion)
        elif opcion == "3":
            opcion_top10_pesados(conexion)
        elif opcion == "4":
            opcion_altura_media(conexion)
        elif opcion == "5":
            opcion_exportar_csv(conexion)
        elif opcion == "6":
            print("¡Hasta luego!")
            break
        else:
            print("Opción no válida.")

    if conexion:
        conexion.close()

```

```
if __name__ == "__main__":  
    main()
```

Cómo utilizar el script

1. Guarda el código en un archivo, por ejemplo `consultar.py`.
2. Asegúrate de que el Script 1 y el Script 2 se han ejecutado al menos una vez para tener los datos locales.
3. Ejecuta desde la terminal:

```
python consultar.py
```

4. Sigue las instrucciones del menú.
5. Si alguna consulta no encuentra el Pokémon en local, automáticamente se usará la PokéAPI (requiere internet). Las opciones 3, 4 y 5 requieren base de datos local; si no existe, mostrarán un aviso.

Comportamiento híbrido

- **Con base de datos:** las búsquedas son instantáneas (offline). Las opciones 3 y 4 ejecutan SQL agregado. La exportación genera el CSV completo.
- **Sin base de datos:** el script informa de la ausencia y solo permite las opciones 1 y 2 usando la API en vivo.

Notas importantes

- La función `buscar_local` acepta tanto un nombre (parcial) como un ID. Si introduces un número, primero intenta encontrarlo por ID exacto.
- La consulta de tipos en local usa la tabla intermedia y agrupa por nombre de Pokémon.
- La opción de tipo en API devuelve todos los Pokémon que tienen ese tipo (incluye formas regionales, etc.). Puede ser una lista muy larga.
- La exportación CSV vuelca la vista `vista_pokemon_tipos`, que ya trae los tipos concatenados.
- Si quieres añadir más funcionalidades (por ejemplo, buscar por habilidad), puedes extender las opciones siguiendo el mismo patrón.

Con este tercer script, dispones de una **Pokédex completa e interactiva** que aprovecha la base de datos local cuando está disponible y recurre a internet solo como respaldo.

Cuestionario y Ejercicios – Pokédex con SQLite

Este documento contiene **10 preguntas de repaso** (cuestionario) y **10 ejercicios prácticos** basados en el proyecto completo: inicialización de la base de datos, descarga de la PokéAPI y consultas offline/online.

Cuestionario

Responde brevemente cada pregunta.

1. ¿Por qué es necesario ejecutar `PRAGMA foreign_keys = ON` al abrir una conexión en SQLite? Explica qué podría ocurrir si no se hace.
2. En la tabla `pokemon_tipo`, ¿por qué la clave primaria es compuesta (`pokemon_id`, `tipo_id`) en lugar de tener una sola columna autoincremental?
3. ¿Qué ventaja tiene usar `INSERT OR IGNORE` en la tabla `tipos` durante la descarga masiva desde la API? ¿Qué pasaría si se usara `INSERT` simple?
4. Observa la función `guardar_pokemon` del Script 2. ¿En qué orden se realizan las inserciones y por qué se debe respetar ese orden?
5. ¿Por qué se crea un índice sobre `tipo_id` en `pokemon_tipo` si la clave primaria ya incluye `pokemon_id` y `tipo_id`? ¿Qué consulta específica se beneficia de él?
6. La vista `vista_pokemon_tipos` utiliza `GROUP_CONCAT`. ¿Qué ocurriría con un Pokémon que tuviera dos tipos? ¿Cómo aparecería en la columna `tipos`?
7. ¿Qué es una tabla intermedia y qué problema de modelado resuelve? Ilustra con el ejemplo de Pokémon y tipos.
8. En el Script 3, cuando no hay base de datos local, ¿cómo obtiene la lista de Pokémon de un tipo la función `opcion_tipo`? ¿Qué URL utiliza?
9. ¿Por qué el trigger `trg_nuevo_pokemon` está definido como `AFTER INSERT` y no como `BEFORE INSERT`? ¿Qué información necesita del nuevo registro?
10. Explica la diferencia entre una vista y una tabla normal. ¿Los datos de la vista se almacenan físicamente en el archivo `.db`?

Ejercicios prácticos

Realiza cada ejercicio modificando o creando pequeños scripts a partir de los ya proporcionados. Usa el archivo `pokemon.db` con datos descargados.

Ejercicio 1 – Verificación del esquema

Crea un script Python que se conecte a `pokemon.db` y muestre por pantalla todas las tablas existentes, así como los índices y las vistas definidos.

Pista: consulta la tabla `sqlite_master`.

```
# Escribe tu código aquí
import sqlite3

conexion = sqlite3.connect("pokemon.db")
cursor = conexion.cursor()
# ... tu consulta ...
conexion.close()
```

Ejercicio 2 – Ampliar la tabla `pokemon`

Añade una columna `color` (TEXT) a la tabla `pokemon`. Luego escribe un script que recorra los Pokémon locales (IDs del 1 al 10), obtenga su color desde la PokéAPI (`pokemon-species`) y actualice la base de datos.

No es necesario modificar el Script 2 completo, solo un pequeño programa adicional.

Ejercicio 3 – Búsqueda parcial mejorada

Modifica la función `buscar_local` del Script 3 para que, si se encuentran varios Pokémon que coinciden con el patrón, muestre todos ellos con un formato abreviado y permita al usuario elegir uno para ver los detalles completos.

Ejercicio 4 – Auditoría con trigger

El trigger `trg_nuevo_pokemon` registra solo el `pokemon_id` y la fecha. Modifícalo para que también guarde el nombre del Pokémon. Si ya está poblada la tabla `log_nuevos`, puedes eliminarla y recrearla con la nueva estructura.

Escribe el nuevo código SQL del trigger y el script Python para aplicarlo.

```
-- Tu nuevo trigger aquí
```

Ejercicio 5 – Estadísticas por tipo

Escribe un programa Python que, sin usar la API, muestre cuántos Pokémon hay de cada tipo (ordenado de mayor a menor). Debe ejecutarse completamente offline.

Usa consultas con JOIN y GROUP BY.

Ejercicio 6 – Exportación selectiva

Amplía la opción 5 (exportar CSV) del Script 3 para que pregunte también por un tipo concreto y exporte únicamente los Pokémon de ese tipo, incluyendo las columnas ID, nombre, altura, peso y tipos concatenados.

Ejercicio 7 – Respaldo online inteligente

En el Script 3, la función `buscar_api` se invoca si no hay datos locales. Modifica el programa para que, **si encuentra el Pokémon en la API, lo inserte automáticamente en la base de datos local** (solo si la base de datos existe), de manera que la próxima consulta ya sea offline.

Ejercicio 8 – Validación de peso

Crea un trigger `BEFORE INSERT` en la tabla `pokemon` que impida insertar un Pokémon con peso negativo o nulo. Si se intenta, debe lanzar un error con `RAISE(ABORT, 'El peso debe ser positivo')`. Pruébalo intentando insertar manualmente un Pokémon con peso -1.

Ejercicio 9 – Comparativa de rendimiento

Con la base de datos poblada con todos los Pokémon, escribe una consulta que muestre el plan de ejecución (**EXPLAIN QUERY PLAN**) al buscar un Pokémon por nombre. Luego borra el índice **idx_pokemon_nombre** (solo temporalmente, en una transacción que hagas rollback) y vuelve a ejecutar el plan. Observa la diferencia y redacta un comentario.

Ejercicio 10 – Función definida por el usuario

Crea una función en Python que convierta el peso de hectogramos (hg) a kilogramos (kg) con un decimal y regístrala en SQLite con **create_function**. Después escribe una consulta SQL que use esa función para mostrar el nombre y el peso en kg de los 5 Pokémon más ligeros.

```
import sqlite3

def peso_kg(hecto):
    return round(hecto / 10, 1)

conexion = sqlite3.connect("pokemon.db")
conexion.create_function("peso_kg", 1, peso_kg)
# ... consulta ...
```